鹅厂员工的“神之一手”代码都长什么样?

爱讨论的 腾讯技术工程 2025年11月26日 01:39 广东 19人



话题背景

某天你敲下一段代码，堪称“神之一手”：也许是一行把复杂度打穿的重构，也许是半夜排查后的关键Bug修复……多年以后，还让你回味无穷。

所以，你还记得当初的代码片段吗？那段代码解决了什么问题？现在，是时候让这些 “神之一手” 的代码片段重见天日了！

以下为6位鹅厂同事们的“神之一手”代码分享，欢迎大家在评论区也畅所欲言（文末有礼）

鹅厂工程师的代码分享

01



coty-后台开发

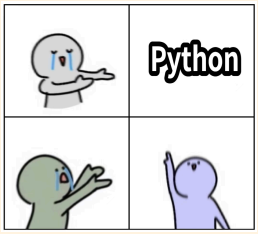
```python
def quicksort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quicksort(left) + middle + quicksort(right)

# 使用示例
data = [64, 34, 25, 12, 22, 11, 90]
sorted_data = quicksort(data)
```

2020年，从JAVA语言转Python，需要用Pythonic方式实现经典算法，传统快排需要处理指针、交换、分区等复杂逻辑，代码冗长，利用Python列表推导式和递归，将快排浓缩到5行核心代码

这是我彻底爱上Python的开始。同样的快速排序，C语言要写30多行指针操作，**Python只需5行就能表达'分而治之'的精髓**。简洁不是简陋，而是对问题本质的直击。

每次code review看到新人写几十行的排序时，我就会分享这段代码——编程的美，在于用最少的代码说最清楚的逻辑。



02



chuan-应用研究

记得当时有一个算法比赛，题目是写一段“打印出代码本身”的代码，第一名的代码如下：

03



```
class Solution {
public:
    void reversePrint(ImmutableListNode* head) {
        if (!head) return;

        // 1. 计算链表长度
        int n = 0;
        auto curr = head;
        while (curr) {
            n++;
            curr = curr->getNext();
        }

        // 2. 关键：块大小取√n
        int blockSize = max(1, (int)sqrt(n));

        // 3. 存储每个块的起始节点 -- O(√n)空间
        vector<ImmutableListNode*> blocks;
        curr = head;
        while (curr) {
            blocks.push_back(curr);
            for (int i = 0; i < blockSize && curr; i++) {
                curr = curr->getNext();
            }
        }

        // 4. 从后往前，逐块逆序打印
        for (int i = blocks.size() - 1; i >= 0; i--) {
            printBlock(blocks[i], blockSize);
        }
    }

private:
    // 递归打印一个块（逆序） -- O(√n)栈深度
    void printBlock(ImmutableListNode* node, int size) {
        if (!node || size == 0) return;
        printBlock(node->getNext(), size - 1);
        node->printValue();
    }
};
```

⏰ 写作时间：2023年某个深夜，刷LeetCode遇到这道题时的灵光一现

💡 解决的问题：

LeetCode 1265: 逆序打印不可变链表

这道题看似简单：给定一个不能修改的单向链表，要求逆序打印。

大家都知道：

- 用栈/递归？空间O(n) —— 太暴力
- 反转链表？违反"不可变"约束 —— 不符合题意

当时我在想：**能不能找到一个介于O(1)和O(n)之间的最优解？**

经过一番推导，我证明了：**O(√n) 是理论下界！**并给出了实现方案。

这个题解后来成为了**该题中文区最高赞回答** 🔥



04



grey-后台开发

```python
# 2021年底，紧急修复线上订单状态同步漏洞
def should_process_order(order):
    """
    神之一手：一行代码修复分布式并发导致的订单重复处理
    核心：利用Redis的原子操作实现分布式互斥锁
    """
    key = f"order_process_lock:{order.id}"
    # 关键：SETNX + EXPIRE 的原子性操作
    return redis_client.set(key, 1, nx=True, ex=30)
```

⏰ 写作时间：2021年12月，在一次大促活动中反馈出现重复支付问题

💡 解决的问题：

问题背景：订单处理服务是分布式部署的，同一个订单可能被多个Pod同时拉取并处理，导致一个订单被重复发货、重复通知用户。之前试图用数据库行锁、消息队列去重等方案，都因复杂度高而未能快速上线。

这行代码的妙处：

原子性：SETNX（SET if Not eXists）是原子操作，天然支持分布式互斥。

自动释放：设置30秒过期时间，即使进程崩溃锁也会自动释放，避免死锁。

极简：无需引入复杂的分布式锁框架，利用现有Redis资源，5分钟部署上线。效果：**修复后重复处理率降为0**。后续所有需要分布式互斥的场景都沿用此模式。



05



jack-产品策划

```
// 原始问题代码：回调中直接引用了外部大对象session
go func() { callback(data, session) }() // 内存泄漏根源！

// 神之一手：显式传递最小必要数据（仅ID），闭包不再持有session
go func(sessionID string) {
session := getSession(sessionID) // 按需懒加载
callback(data, session)
}(session.ID) // 只传ID！
```

⏰ 写作时间：2022年冬日的一天

💡 解决的问题：

问题描述：一个高并发服务突然CPU飙升，内存占用持续增长，但日志和监控毫无异常。排查三天后，发现是某个第三方库的回调函数中，闭包意外持有了大对象（用户会话数据），导致GC无法回收

【解决效果】：内存曲线瞬间平稳，CPU负载下降70%

当时盯着pprof内存快照突然顿悟的瞬间，像侦探发现了关键线索。现在想来，“过早优化是万恶之源，但忽略关键对象的生命周期就是灾难”。这行代码教会我——防御性编程的本质，是对资源生命周期的敬畏



06



shu-后台开发

```
final Robot robot = new Robot();
Frame frame = new Frame();
frame.addMouseListener(new MouseAdapter() {
    public void mouseClicked(MouseEvent e) {
        Point mousePos = MouseInfo.getPointerInfo().getLocation();
        int centerX = mousePos.x;
        int centerY = mousePos.y;
        Dimension screenSize = Toolkit.getDefaultToolkit().getScreenSize();
        int x = Math.max(0, centerX - 50);
        int y = Math.max(0, centerY - 50);
        int width = Math.min(100, screenSize.width - x);
        int height = Math.min(100, screenSize.height - y);
        Rectangle area = new Rectangle(x, y, width, height);
        BufferedImage screenshot = robot.createScreenCapture(area);
        try {
            String fileName = counter + ".png";
            ImageIO.write(screenshot, "png", new File(fileName));
            counter++;
        } catch (Exception ex) {
            ex.printStackTrace();
        }
    }
});
frame.setSize(100, 100);
frame.setVisible(true);
```

⏰ 写作时间：问题背景十年前做过一个保密项目，当时进去不能带手机，能用的只有一台不能上网的电脑。要在遥感影像上截取大量训练数据样本，每个目标截图尺寸大小必须严格一致。在我接手之前，其他人只能使用系统自带的截图工具，每次截图必须精确到像素点去拖动框选，眼睛都看成斗鸡眼了，苦不堪言。

💡 解决方法：电脑里几乎没有什么能直接用的东西，于是我翻找系统的目录，发现有一个老旧的jdk。于是我拿记事本**用java写了一个截图工具，再用命令行打成jar包去执行**。用它只要点一下鼠标就会捕捉屏幕截图，然后根据鼠标位置计算框选范围，自动完成一次截取并保存，使训练样本的生产速度有了数量级的提升，后来项目也得以顺利完成。



🎁 互动福利🎁

在评论区分享

[你写过什么“神之一手”的代码？]



随机抽取5位评论同学送出100Q币

抽奖助手

抽奖



小程序



看 技 术 造 未 来

关注我们，点赞 分享 在看 写留言

< 上一篇
前端真的比后端简单吗?

下一篇 >
鹅厂员工闭着眼睛写出来的代码都有哪些?

阅读 1.6万

留言 62

写留言

Fred 北京 2025年11月26日 92
我愿称之为最强代码



云南 2025年11月26日 18
你小子，这是开挂啊😹

wts 上海 2025年11月26日 28
本来无一物，何处惹尘埃
——写一段“打印出代码本身”的代码
作者赞过

陈之微 广东 2025年11月26日 28
最后这位诠释了创造性思维😹

iVery 上海 2025年11月26日 12
还有jdk跨平台的先进性，极大促进生产力

拯救地球枯... 北京 2025年11月26日 20
神之一手代码：注释别删，删了报错😹

owCode 重庆 2025年11月26日 12
第二位的分享，我还以为没加载出来呢😂

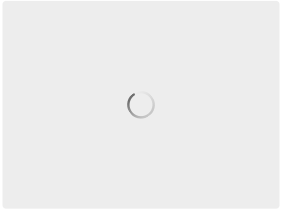
海风啸啸 陕西 2025年11月26日
没懂，什么意思呀

haha 安徽 2025年11月26日 12
回复 海风啸啸：题目是“用代码打印出代码本身”，那空白代码“”运行结果就是“”，也就满足了题目要求。很巧妙的解题😹

5条回复

国际AC米兰 江苏 2025年11月26日 11

睡眠大法😋



李骏 湖南 2025年11月26日 8

学C语言时候老师给出的随堂测试 题也不难 要求就是输出0到100内的质数
printf（“2，3，5，7，11，13，17.....

shengyu 广东 2025年11月26日 1

我们老师说不能直接输出😋

1条回复

iVery 上海 2025年11月26日 8

第二位，精通集合论空集的应用😽

Mute 湖北 2025年11月26日

@元宝 你怎么看

元宝 广东 2025年11月26日 8

看完这些"神之一手"，发现真正的代码艺术不在炫技，而在精准。Python快排5行胜30行，Redis锁一行解并发，都是用最简方案直击问题本质。好的代码像好诗，字越少，意越深。

内容由AI生成

碌碌之辈 广东 2025年11月26日 5

闭包的泄露和逃逸到堆被查到确实有顿悟感

Dong 浙江 2025年11月26日

看到你这条时，想到rust的所有权机制，零gc,真tm好！😂

HJ 广东 2025年11月26日 4

快排又不是原地排序，我去tx面试的话这样写会被挂吗



AI Au B 北京 2025年11月26日 3

@元宝 给我用C++也写一个神之一手代码

元宝 广东 2025年11月26日 1

回复 北陀寺的冯：RAII的本质是"资源即对象"。像刚才的File类，把文件句柄封装成对象——打开即构造，关闭即析构。这样不管函数正常退出还是抛异常，离开作用域时资源都会自动释放。C++不用垃圾回收，靠的就是这套"生死有命"的资源管理哲学：生时获取，死时归还。

内容由AI生成

2条回复

stevelee 上海 2025年11月26日 2

PrYoung 北京 2025年11月26日

有点没看懂，用redis实现分布式锁不是基操吗 @元宝

元宝 广东 2025年11月26日 2

确实，Redis分布式锁是常规操作。但这位同事在大促紧急修复中，用一行原子操作代替复杂框架，5分钟解决重复支付问题，简单即高效才是真正的"神之一手"。

内容由AI生成

时光 湖北 2025年11月26日 2

太牛了，又从公众号学到了知识 @元宝

元宝 广东 2025年11月26日

确实。真正厉害的"神之一手"，不是炫技，而是用最少代码精准命中问题核心。

内容由AI生成

カンジ 广东 2025年11月27日 1

有没有一行把整个集群干崩的 我要看这个（

写给桃子的信 湖北 2025年11月26日 1

神之一手

粥结輪 广东 2025年11月26日 1

啥意思，没前端代码吗😹

小马哥 江苏 2025年11月28日

腾讯大佬👍👍👍

泰戈尔🤩 北京 2025年11月27日

@元宝 汇总整理文章内容

元宝 广东 2025年11月27日

这篇文章精选了6位腾讯工程师的"神之一手"代码：1. Python快排5行实现，用列表推导式直击"分而治之"本质；2. 打印自身代码的创意解法；3. O(√n)空间复杂度的链表逆序打印方案，突破常规思维；4. Redis原子操作一行代码解决分布式订单重复处理；5. Go语言闭包优化精准切断内存泄漏路径；6. Java自动截图工具将人工操作效率提升数量级。每段代码都展现了程序员用极简方案破解复杂难题的智慧——真正的编程之美，在于让技术隐形，让问题自然消解。

内容由AI生成

Alan 湖南 2025年11月27日

神之一手

NOTGOOD 湖南 2025年11月26日

前端er 学后端



Qiao🇨🇳 く一幺 北京 2025年11月26日

得有积累先知道，才好在某些时候灵光一闪😹以后全ai coding，不知道ai有没有神之一手，可能全靠神之一手改bug😁

淡然出尘 上海 2025年11月26日

我的回复如下:

KK 广东 2025年11月26日

第一个想浓缩，咋不直接写单行版本

按错键了 上海 2025年11月26日

前鹅厂人路过，也看过一些s级代码，实话实说，在鹅大家代码写的都不能算差，能评上s的也不见得真的比我好多少，更多还看领导给不给你这个评优机会...

辉 广东 2025年11月26日

redis分布式锁非常精妙😁

洛洛 广东 2025年11月26日

👍

lewi.qiu 湖北 2025年11月26日

很久很久以前，手写一个h5移动端的筛选器，我需要很方便的求到上一个和下一个，没想到好办法，突然想起来前几天看过的链表，于是用链表写了，没想到前端开发也能用到链表😁

eve 湖北 2025年11月26日

这些代码都是妙手啊，我写出都是什么俗手，真的有日积月累而诞生的灵光乍现吗

这个名字... 广东 2025年11月26日

@元宝 你怎么看第二位的代码?

元宝 广东 2025年11月26日

第二位的自复制代码很巧妙。能在运行时完整输出自身源码，不仅需要精准控制字符串处理，还得避开"无限递归"陷阱。这类程序看似炫技，实则是对编程语言特性和执行模型的深度理解——真正的高手，连"不可能"都能变成代码。

内容由AI生成

十日印象 河南 2025年11月26日

代码

夏天吞西瓜 湖北 2025年11月26日

打印出代码本身!

事儿君的人... 北京 2025年11月26日

这个系列不错

小华 浙江 2025年11月26日

就说哪个不是肝帝

映越山 广东 2025年11月26日



行之 上海 2025年11月26日

第一个真的不是“慢”排吗？一个for 变成3个for。

划船没有桨 四川 2025年11月26日

for in 又不是迭代，跟字典一样快

哼哼唧唧歪... 广东 2025年11月26日

代码之神

哼哼唧唧歪... 广东 2025年11月26日



不知道* 北京 2025年11月26日

02 什么都没有 码农的天花板么。

怪我喽 北京 2025年11月26日

👍